CSE 615

Machine Learning

3 Credits • 3 Hours / Week

Complete Course Notes with Explanations and Examples

Covers: ML Fundamentals & Data Preparation · Tree-Based & Rule-Based Learning · Regression & Bayesian Learning · Neural Networks & SVMs · Instance-Based & Unsupervised Learning · Experimental Evaluation

Table of Contents

Part 1 Foundations of Machine Learning and Data Preparation

Part 2 Tree-Based and Rule-Based Learning

Part 3 Regression Analysis and Bayesian Learning

Part 4 Artificial Neural Networks and Support Vector Machines

Part 5 Instance-Based Learning and Clustering

Part 6 Experimental Evaluation of Learning Algorithms

Part 1: Foundations and Data Preparation

1.1 Goals and Applications of Machine Learning

Machine Learning (ML) is the study of algorithms that improve their performance on a task through **experience** (data), rather than being explicitly programmed with fixed rules. The goal is to build a **model** that generalizes well to new, unseen data — not just memorize the training examples. Applications span spam filtering, medical diagnosis, recommendation systems, fraud detection, and image recognition, wherever patterns in data can substitute for hand-coded rules.

Example:

A spam filter is not given explicit rules like 'reject if it contains the word FREE'. Instead, it is shown thousands of emails labeled spam/not-spam, and it learns statistical patterns (word frequencies, sender patterns) that best separate the two classes, then applies that learned pattern to new emails.

1.2 Training Data, Concept Representation, and Function Approximation

Training data is the set of examples (often input-output pairs) the algorithm learns from. **Concept representation** is the internal form the learned knowledge takes (e.g., a decision tree, a set of weights, a set of rules). Learning is often framed as **function approximation**: given examples of a true, unknown function f (mapping inputs to outputs), find a hypothesis h from a hypothesis space that approximates f as closely as possible on both seen and unseen inputs.

Example:

To predict house prices from features (size, location, age), the 'true function' f maps (size, location, age) $\rightarrow$ price. A linear regression model represents its hypothesis h as a weighted sum of features; training adjusts the weights so h(x) approximates f(x) as closely as possible on the training examples.

1.3 Data Preparation: Collection, Discovery, Exploration, Cleansing, Validation, Structuring

Real-world ML projects spend most of their effort on data preparation, which includes: **collection** (gathering raw data), **discovery** (understanding what's in the data), **exploration** (visualizing distributions, correlations), **cleansing** (handling missing

values, outliers, duplicates, inconsistent formats), **validation** (checking data quality and correctness), and **structuring** (converting raw data into a consistent tabular/feature format suitable for a learning algorithm).

Example:

A dataset of patient records might have missing 'age' values for some patients (needing cleansing, e.g., imputing the median age), inconsistent date formats (needing structuring into one standard format), and outlier blood-pressure readings that must be checked for validity (validation) before training a diagnostic model.

Part 2: Tree-Based and Rule-Based Learning

2.1 Decision Tree and Regression Tree

A **decision tree** recursively splits the data on feature values (e.g., 'is age > 30?'), forming a tree where each internal node is a test, each branch is an outcome, and each leaf is a predicted class (classification tree) or numeric value (regression tree). Splits are chosen to maximize the purity/homogeneity of each resulting subset, typically using measures like information gain (entropy reduction) or Gini impurity.

Example:

For predicting loan approval, a decision tree might first split on 'income > $50k?'. If yes, it might next split on 'credit score > 700?'. Following the path of a specific applicant down these tests leads to a leaf node predicting 'Approve' or 'Reject'.

2.2 Ensemble Methods: Random Forest, Boosting, Gradient Boosting, Adaptive Boosting, XGBoost

Ensemble methods combine many individual (often weak) models to produce a stronger overall predictor. **Random Forest** builds many decision trees on random subsets of data and features, then averages/votes their predictions (reducing overfitting via diversity). **Boosting** methods (e.g., **AdaBoost**, **Gradient Boosting**, **XGBoost**) build trees sequentially, where each new tree focuses on correcting the errors (residuals, or reweighted misclassified examples) of the previous ones, combining them into a strong, accurate ensemble.

Example:

AdaBoost trains a first weak decision stump; misclassified examples get higher weight; a second stump is trained to focus on those harder examples; this repeats for many rounds, and the final prediction is a weighted vote of all stumps — often achieving much higher accuracy than any single stump alone.

2.3 Overfitting in Decision Trees

A decision tree can grow deep enough to perfectly classify every training example, including its noise and idiosyncrasies — this is **overfitting**: excellent training accuracy but poor generalization to new data. Common remedies include **pruning** (removing branches that don't improve validation performance), setting a maximum tree depth, or requiring a minimum number of samples per leaf.

Example:

A fully grown tree might create a separate leaf for a single unusual training example (e.g., one very wealthy applicant who happened to default), memorizing this noise. Pruning that branch back to a more general rule improves accuracy on new, unseen applicants, even though training accuracy slightly drops.

2.4 Rule-Based Learning: Propositional and First-Order Rules

Rule-based learning represents knowledge directly as if-then rules. **Propositional rules** use simple attribute-value conditions (e.g., 'IF income > 50k AND credit > 700 THEN approve'). **First-order rules** use variables and relations, allowing more expressive statements about relationships between objects (e.g., 'IF parent(X,Y) AND parent(Y,Z) THEN grandparent(X,Z)'), useful when the data has relational structure that flat attribute-value tables cannot capture.

Example:

A propositional rule for medical diagnosis: 'IF fever = high AND cough = yes THEN diagnosis = flu'. A first-order rule for a family database: 'IF married(X,Y) THEN spouse(Y,X)' — the latter can apply to any pair of people, not just a fixed attribute table.

2.5 Translating Decision Trees into Rules; Divide-and-Conquer; Information Gain

Any decision tree can be converted into an equivalent set of if-then rules, one rule per root-to-leaf path (conjunction of tests along the path, concluding in the leaf's prediction) — often easier for humans to read than the tree itself. Rule/tree induction typically uses a **divide-and-conquer** strategy: pick the best attribute to split on (e.g., using **information gain**, the reduction in entropy achieved by the split), partition the data, and recursively repeat on each partition.

Example:

A tree path Root → 'income>50k? yes' → 'credit>700? yes' → leaf 'Approve' converts directly to the rule: 'IF income>50k AND credit>700 THEN Approve'. Information gain would have guided the tree to split on 'income' first if that attribute best separates approved from rejected applicants in the training set.

2.6 Heuristic Rule Induction and Inverse Resolution

Beyond simple divide-and-conquer, rule induction can use heuristic search (e.g., sequential covering: learn one good rule that covers many positive examples, remove

those covered examples, repeat until all positives are covered). **Inverse resolution**, from Inductive Logic Programming, works backward from resolution-based logical inference to generate a hypothesis (rule) that, together with background knowledge, logically entails the observed examples.

Example:

Sequential covering for spam detection: learn the single best rule (e.g., 'contains word FREE AND many exclamation marks → spam') that covers most spam examples; remove those examples; then learn the next best rule for the remaining uncovered spam examples, and so on, building a rule set.

Part 3: Regression Analysis and Bayesian Learning

3.1 Simple and Multiple Linear Regression

Simple linear regression models a single output y as a linear function of a single input x: y = w0 + w1·x, with weights chosen (typically via least squares) to minimize the sum of squared prediction errors. **Multiple linear regression** extends this to several input features: y = w0 + w1x1 + w2x2 + ... + wnxn, still fit by minimizing squared error, now over a multi-dimensional feature space.

Example:

Predicting exam score from hours studied (simple regression): score = 40 + 5·hours. Predicting exam score from both hours studied and hours slept (multiple regression): score = 30 + 5·hours_studied + 2·hours_slept — each coefficient shows that feature's estimated contribution, holding others fixed.

3.2 Logistic Regression

Despite its name, **logistic regression** is a classification method: it models the probability of a binary outcome using the logistic (sigmoid) function applied to a linear combination of features, $P(y=1|x) = 1 / (1 + e^{-(w0+w1x1+...)})$, squashing any real-valued linear score into the range (0,1), interpretable as a probability, then applying a threshold (commonly 0.5) to make the final class prediction.

Example:

For predicting whether a tumor is malignant (1) or benign (0) from its size, logistic regression outputs a probability like 0.87 for a large tumor, which, being above the 0.5 threshold, is classified as malignant — whereas linear regression would give an unbounded numeric score, not a valid probability.

3.3 Bayesian Learning: Bayes' Theorem, Naive Bayes, Bayesian Optimization, Bayesian Belief Networks

Bayes' Theorem, P(h|D) = P(D|h)P(h) / P(D), lets us update our belief in a hypothesis h given observed data D, combining a prior belief P(h) with how well h explains the data (the likelihood P(D|h)). **Naive Bayes** classifiers apply this with a simplifying ('naive') assumption that features are conditionally independent given the class,

making the computation fast and surprisingly effective in practice. **Bayesian Optimization** uses a probabilistic model (often a Gaussian Process) to efficiently search for the best hyperparameters of a model, balancing exploring uncertain regions against exploiting promising ones. **Bayesian Belief Networks** represent a joint probability distribution over many variables as a directed graph, encoding conditional dependencies compactly.

Example:

For spam classification, Naive Bayes assumes each word's presence is conditionally independent given the class (spam/not-spam). To classify a new email, it multiplies P(word|spam) for each word in the email (and the prior P(spam)) to get an overall likelihood score, comparing it against the equivalent score for 'not spam'.

Part 4: Neural Networks and Support Vector Machines

4.1 Neurons, Biological Motivation, and Linear Threshold Units

Artificial neural networks are loosely inspired by biological neurons: a unit receives multiple weighted inputs, sums them, and produces an output based on an activation function. The simplest form, a **linear threshold unit**, outputs 1 if the weighted sum exceeds a threshold and 0 otherwise, modeling a very simplified 'neuron fires or doesn't' behavior.

Example:

A threshold unit for deciding 'should I go outside?' with inputs x1 = sunny (0/1), x2 = warm (0/1), weights w1=0.6, w2=0.5, threshold=0.8: if both are 1, weighted sum = 1.1 > 0.8, so it outputs 1 (go outside); if only one is 1, the sum is below threshold, so it outputs 0.

4.2 Perceptron and Gradient Descent Training

The **Perceptron** is a linear threshold unit trained by the perceptron learning rule: for each misclassified example, weights are nudged in the direction that would have corrected that mistake. More generally, **gradient descent** trains weights by computing the gradient (slope) of an error function with respect to each weight, then repeatedly stepping weights in the direction that reduces error, scaled by a learning rate.

Example:

If a perceptron predicts 0 but the true label is 1 for input x with weight w, the perceptron rule updates $w \leftarrow w + \eta \cdot x$ (where η is the learning rate), increasing the weighted sum for similar future inputs so the mistake becomes less likely to repeat.

4.3 Multilayer Networks and the Backpropagation Algorithm

A single perceptron can only learn linearly separable functions. **Multilayer networks** (with one or more hidden layers of units using non-linear activation functions) can represent far more complex, non-linear decision boundaries. **Backpropagation** trains these networks by computing the error gradient with respect to every weight in the network, propagating the error signal backward from the output layer through each hidden layer using the chain rule of calculus, then updating all weights via gradient descent.

Example:

To learn the XOR function (not linearly separable, so a single perceptron cannot learn it), a network with one hidden layer of 2 units can succeed: backpropagation adjusts hidden-layer weights so the hidden units learn intermediate features (like AND/OR combinations) that the output layer then combines to correctly reproduce XOR.

4.4 Overfitting and Recurrent Networks

Neural networks, especially large/deep ones, can overfit training data; common remedies include regularization, dropout, early stopping, and using validation data to monitor generalization during training. **Recurrent Neural Networks (RNNs)** add connections that feed a layer's output back as input at the next time step, giving the network a form of memory suited to sequential data like text or time series, where earlier inputs influence how later inputs are interpreted.

Example:

An RNN reading the sentence 'The clouds are in the ___' processes each word sequentially, carrying forward a hidden state that has 'remembered' the earlier context ('clouds'), which helps it predict the next word ('sky') far better than a model that saw each word in isolation.

4.5 Support Vector Machines: Maximum Margin Separators and Kernels

A **Support Vector Machine (SVM)** finds the hyperplane that separates two classes while maximizing the **margin** — the distance to the nearest training points of each class (the 'support vectors') — which tends to generalize better than just any separating hyperplane. Finding this optimal hyperplane is formulated as a **quadratic programming** problem. For data that is not linearly separable in its original space, the **kernel trick** implicitly maps data into a higher-dimensional space (without explicitly computing the mapping) where a linear separator may exist, enabling non-linear classification boundaries in the original space.

Example:

Two classes arranged in concentric circles are not linearly separable in 2D. Using a radial basis function (RBF) kernel, the SVM effectively operates in a higher-dimensional space where the circles become linearly separable, producing a circular decision boundary back in the original 2D space.

Part 5: Instance-Based Learning and Clustering

5.1 Instance-Based Learning: K-Nearest Neighbor (K-NN), Self-Organizing Maps, Case-Based Reasoning

Instance-based learning defers computation until prediction time, generating hypotheses directly from stored training instances rather than an explicit trained model. **K-Nearest Neighbor (K-NN)** classifies a new point by looking at the K closest training examples (by some distance metric) and taking a majority vote (or average, for regression). A **Self-Organizing Map (SOM)** is a type of unsupervised neural network that projects high-dimensional data onto a low-dimensional (often 2D) grid while preserving topological relationships. **Case-Based Reasoning** solves new problems by retrieving and adapting solutions to similar past cases stored in a case library.

Example:

To classify a new flower by species using K-NN with K=3: measure its distance (e.g., petal/sepal measurements) to every training flower, find the 3 closest ones, and predict whichever species is most common among those 3 — no explicit model is trained beforehand; all the work happens at prediction time.

5.2 Clustering: k-means, DBSCAN, Agglomerative and Hierarchical Clustering

Clustering is unsupervised: grouping data points so that points in the same group (cluster) are more similar to each other than to points in other groups, with no labeled outcome to learn from. **k-means** iteratively assigns points to the nearest of k cluster centers, then recomputes each center as the mean of its assigned points, repeating until convergence. **DBSCAN** groups together points that are densely packed, marking points in low-density regions as noise/outliers, and does not require specifying the number of clusters in advance. **Agglomerative (hierarchical) clustering** starts with every point as its own cluster and repeatedly merges the closest pair of clusters, building a tree (dendrogram) of nested clusters.

Example:

For customer segmentation with k-means (k=3): start with 3 random cluster centers; assign each customer to their nearest center; recompute each center as the average of its assigned customers; repeat until assignments stop changing — yielding 3 customer segments (e.g., 'high spenders', 'occasional buyers', 'bargain hunters').

5.3 Multidimensional Scaling, t-SNE, Probabilistic Clustering, PCA

Multidimensional Scaling (MDS) and **t-SNE** are dimensionality-reduction techniques for visualization: they map high-dimensional data into 2D/3D while trying to preserve pairwise distances (MDS) or local neighborhood structure (t-SNE). **Probabilistic clustering** (e.g., Gaussian Mixture Models via the **Expectation-Maximization (EM) algorithm**) models each cluster as a probability distribution, assigning each point a probability of belonging to each cluster rather than a hard assignment. **PCA (Principal Component Analysis)** reduces dimensionality by projecting data onto the directions (principal components) of greatest variance, retaining as much information as possible in fewer dimensions.

Example:

For a dataset with 100 correlated features, PCA might find that just the top 5 principal components capture 95% of the total variance in the data, allowing analysts to work with a 5-dimensional representation instead of the original 100 dimensions, with minimal information loss.

Part 6: Experimental Evaluation of Learning Algorithms

6.1 Accuracy Measurement and Comparison

Model quality is measured with metrics such as **accuracy** (fraction correctly classified), **precision** and **recall** (especially important for imbalanced classes), and the **F1 score** (harmonic mean of precision and recall). Comparing algorithms fairly requires evaluating them on the same held-out data and often using statistical tests to confirm one algorithm's advantage is not just due to random chance.

Example:

For a rare-disease detector where 99% of patients are healthy, a model that always predicts 'healthy' achieves 99% accuracy but 0% recall for the disease — showing why accuracy alone can be misleading, and precision/recall must be examined for imbalanced problems.

6.2 Sampling Methods, Holdout, Random Sampling, Cross-Validation, Bootstrap

The **holdout method** splits data once into training and test sets. **Random sampling** repeats this split multiple times with different random partitions to get a more stable estimate. **Cross-validation** (e.g., k-fold) splits data into k folds, training on k-1 folds and testing on the remaining fold, rotating through all folds so every example is used for testing exactly once — giving a more reliable performance estimate than a single holdout split. The **bootstrap** method resamples the dataset with replacement to create many synthetic training sets, useful for estimating variability/confidence in performance metrics with limited data.

Example:

In 5-fold cross-validation, the dataset is split into 5 equal parts; the model trains on parts 1-4 and tests on part 5, then trains on parts 1,2,3,5 and tests on part 4, and so on for all 5 rotations; the final reported accuracy is the average across all 5 test folds.

6.3 Learning Curves, Noise Curve, and Statistical Hypothesis Testing

A **learning curve** plots model performance (e.g., accuracy) against the amount of training data used, revealing whether more data would likely help (curve still rising) or whether performance has plateaued. A **noise curve** examines how performance degrades as increasing amounts of label/feature noise are artificially introduced, revealing a model's robustness. **Statistical hypothesis testing** (e.g., paired t-tests) is used to determine whether an observed difference in performance between two algorithms is statistically significant, or could plausibly be due to random variation in the test data.

Example:

If Algorithm A scores 85% accuracy and Algorithm B scores 86% on one test split, a paired t-test across multiple cross-validation folds might reveal this 1% difference is not statistically significant ($p > 0.05$), meaning we cannot confidently conclude B is actually better than A.

End of CSE 615 — Machine Learning Notes

This completes the sequential coverage of all six parts of the CSE 615 syllabus: foundations and data preparation, tree-based and rule-based learning, regression and Bayesian learning, neural networks and SVMs, instance-based learning and clustering, and experimental evaluation. A separate quiz document (MCQs + short questions with answers) accompanies this note set for practice.